Virtual Memory

Operating Systems Ch 10

Week 12

Today's Schedule

Hour	Content
1st	**Quiz (Beginning)** → Theory Lecture (Part 1)
2nd	Theory Lecture (Part 2)
3rd	Hands-on Lab

Part 1

Background / Demand Paging

Virtual Memory Overview

Execute programs without loading them entirely into memory

- Many parts of a program are not used during execution
  - Error handling code, optional features, large declared arrays
- What if only part of a program needs to be in memory to run?
  - Can execute programs larger than physical memory
  - Accommodate more processes simultaneously
  - Faster process execution due to reduced I/O

Definition of Virtual Memory

Separation of Logical memory and Physical memory

- **Virtual Memory**: A technique that separates logical memory from physical memory
- Provides programmers with a very large virtual memory space
- Actual physical memory can be small
- Implementation method: **Demand Paging** (most common)
- Additional benefits
  - Memory sharing through shared libraries
  - Inter-process communication through shared memory
  - Efficient process creation through fork()

Virtual Address Space

The logical memory space as seen by a process

- Each process has an independent virtual address space
- Typical structure: starts from address 0, appears contiguous
- Actual physical memory page frames may be non-contiguous
- **Sparse Address Space**
  - Holes exist between Code (text), Data, Heap, and Stack
  - Heap grows upward, Stack grows downward
  - Holes do not occupy actual physical memory
  - Shared libraries are mapped to the middle region

Shared Library and Virtual Memory

Library sharing through virtual memory

- Multiple processes can share libraries such as the standard C library
- Mapped into each process's virtual address space, but physical pages are shared
- Libraries are typically mapped as read-only
- Shared memory: memory region sharing for inter-process communication
- Page sharing during fork() accelerates process creation

Demand Paging

Load pages into memory only when needed

- **Lazy Swapper (Pager)**
  - Does not swap in a page until it is actually needed
  - Does not load all pages when a process starts
  - Unused pages do not occupy memory
- Advantages
  - Reduced I/O: no unnecessary page loading
  - Memory savings: only needed pages are loaded
  - Faster response time: quicker process startup
  - Can accommodate more processes

Valid / Invalid Bit

Hardware support to distinguish whether a page is in memory

- A valid-invalid bit is added to each page table entry
- **Valid**: the page is legitimate and present in memory
- **Invalid**: the page is either invalid or resides on disk
- All entries are set to invalid when a process starts
- Changed to valid when a page is loaded into memory
- Accessing an invalid page triggers a **page fault** trap
- If a page is not used, there is no impact

Page Fault Handling Process

Step-by-step procedure when an invalid page is accessed

1. Check the valid/invalid bit in the page table during memory access
2. If invalid, a **page fault trap** occurs (control transfers to OS)
3. OS checks: determines whether it is an invalid address or a page on disk
4. Find a free frame (from the free-frame list)
5. Load the page from disk into the found frame (I/O scheduling)
6. Update the page table: record the frame number, change to valid
7. **Restart the instruction** that caused the trap

Pure Demand Paging

An extreme demand paging strategy

- Start without loading **any page** into memory
- Page fault occurs from the very first instruction
- Page fault rate decreases due to **locality of reference**
- No more faults once all needed pages are loaded
- Hardware required for demand paging
  - **Page table**: supports valid-invalid bit
  - **Secondary memory** (swap space): stores pages not in memory
  - **Instruction restart** capability: re-execute instruction after page fault

Difficulty of Instruction Restart

Restarting an instruction after a page fault is not always easy

- Simple case: fault during instruction fetch - just re-fetch
- Complex case: ADD A, B, C
  1. Instruction fetch & decode
  2. Fetch A
  3. Fetch B
  4. Add A + B
  5. Store to C - what if a page fault occurs here?
  - Must restart the entire process from the beginning
- IBM MVC instruction problem: page boundary during copying up to 256 bytes
  - Solution 1: Pre-check both end addresses for potential faults
  - Solution 2: Save original values in temporary registers, restore on fault

Free-Frame List

An OS data structure for managing free frames

- OS maintains a **free-frame list** (pool of available frames)
- Frames are allocated from this list when a page fault occurs
- **Zero-fill-on-demand**: initialize frames to 0 before allocation
  - Prevents exposure of previous process data (security)
- All available memory is registered in the free-frame list at system boot
- **Page replacement** is needed when free frames run low

Demand Paging Performance

Impact of page faults on performance

- **EAT (Effective Access Time)** calculation
  - p = page fault rate (0 <= p <= 1)
  - EAT = (1 - p) x memory access time + p x page fault time
- Page fault service time components (3 major tasks)
  1. Page-fault interrupt handling
  2. Page read (I/O)
  3. Process restart

Demand Paging Performance Calculation Example

Performance impact with concrete numbers

- memory access time = 200ns, page fault time = 8ms

```
EAT = (1 – p) x 200 + p x 8,000,000
    = 200 + 7,999,800 x p
```

- p = 1/1,000 (1 fault per 1000 accesses)
  - EAT = 8,200ns - **40x slower!**
- To keep performance degradation within 10%:
  - 220 > 200 + 7,999,800 x p
  - p < 0.0000025 (approximately 1/400,000)
- **Conclusion**: page fault rate must be kept extremely low to ensure performance

Swap Space Utilization

Performance differences based on page storage location

- Swap space I/O is faster than file system I/O
  - Larger block allocation, no file lookup needed
- Strategy 1: Copy entire file to swap space, then demand page
- Strategy 2: Initially from file system, write to swap space on replacement
  - Approach used by Linux and Windows
- Binary executable: demand paging directly from file system
  - Not modified, so can simply overwrite on replacement
  - Only **anonymous memory** (stack, heap) uses swap space

Part 2

Copy-on-Write / Page Replacement

Copy-on-Write

Optimization technique for fork()

- Parent and child **share the same pages** during fork()
- Pages are not copied immediately
- A page is copied only when either side **writes** to it
- Efficient since only written pages are copied
- Operation process
  1. fork() call: copy parent's page table, pointing to the same frames
  2. Set shared pages to **read-only**
  3. A **protection fault** occurs on a write attempt
  4. OS copies the page to a new frame
  5. Update both page tables, allow write

Copy-on-Write Details

Non-modifiable pages and vfork()

- Non-modifiable pages (executable code) do not need COW - always shared
- Windows, Linux, and macOS all use COW
- **vfork()** (virtual memory fork)
  - Parent **suspends** until child terminates (or calls exec)
  - Pages are shared but not copied
  - Child directly uses parent's address space (dangerous!)
  - Used when the child is certain to call exec()
  - Very fast since no page copying occurs at all
  - Used in UNIX command-line shell implementations

Need for Page Replacement

The problem when there are no free frames

- **Over-allocation**: situation where all memory is in use
- Page fault occurs - page location confirmed on disk
- But **there are no free frames!**
- Solution: evict one of the pages already in memory
  - This process is **page replacement**

Basic Page Replacement Procedure

Victim frame selection and replacement process

1. Locate the needed page on disk
2. Find a free frame
   - If available, use it
   - If not, select a victim frame using a **page replacement algorithm**
   - Write the victim frame back to disk (if modified)
3. Load the new page into the freed frame
4. Update the page table
5. Restart the process

- **Dirty Bit (Modify Bit)**
  - Set to 1 for pages modified in memory
  - If dirty bit = 0, disk write can be skipped
  - **Can reduce I/O by half**

Significance of Page Replacement

Completing demand paging

- Page replacement **fully separates logical memory from physical memory**
- A 20-page process can run with only 10 frames
- Two key problems must be solved
  1. **Frame allocation algorithm**: how many frames to allocate to each process?
  2. **Page replacement algorithm**: which page to replace?
- Goal: choose an algorithm that **minimizes the page fault rate**

Reference String

Evaluation criteria for page replacement algorithms

- Run algorithms against a specific **reference string**
- Count and compare page fault occurrences
- Extract only page numbers from the address sequence
- Example: address sequence (page size = 100 bytes)

```
0100, 0432, 0101, 0612, 0102, 0103, ...
→ Reference string: 1, 4, 1, 6, 1, 6, 1, 6, 1, 6, 1
```

- Increasing the number of frames generally reduces page faults
- Reference string used in subsequent examples:

```
7, 0, 1, 2, 0, 3, 0, 4, 2, 3, 0, 3, 2, 1, 2, 0, 1, 7, 0, 1
```

FIFO Page Replacement

The simplest replacement algorithm

- Replace the **oldest** page (First-In, First-Out)
- Simple implementation: managed with a FIFO queue
- New page arrives - insert at queue tail
- Replacement needed - replace the page at queue head

FIFO Operation Example

3 frames, reference string: 7 0 1 2 0 3 0 4 2 3 0 3 2 1 2 0 1 7 0 1

Ref	Frame1	Frame2	Frame3	Fault
7	7	-	-	F
0	7	0	-	F
1	7	0	1	F
2	**2**	0	1	F
0	2	0	1	
3	2	**3**	1	F

FIFO Operation Example (cont.)

Ref	Frame1	Frame2	Frame3	Fault
0	**0**	2	3	F
3	0	2	3	
2	0	2	3	
1	0	**1**	3	F
2	0	1	**2**	F
0	0	1	2	
1	0	1	2	

Belady's Anomaly

A fundamental limitation of FIFO

- Increasing the number of frames is expected to reduce page faults
- However, in FIFO, **page faults can increase even with more frames!**
- Reference string: **1, 2, 3, 4, 1, 2, 5, 1, 2, 3, 4, 5**

Frames	Page Faults
3 frames	**9** faults
4 frames	**10** faults

- 4 frames causes **more** page faults than 3 frames!
- This phenomenon is called **Belady's Anomaly**
- Demonstrates a fundamental limitation of the FIFO algorithm

Belady's Anomaly Detailed Example

Reference string: 1, 2, 3, 4, 1, 2, 5, 1, 2, 3, 4, 5 (3 frames)

Ref	F1	F2	F3	Fault
1	1	-	-	F
2	1	2	-	F
3	1	2	3	F
4	**4**	2	3	F
1	4	**1**	3	F
2	4	1	**2**	F

Belady's Anomaly Detailed Example (4 frames)

Reference string: 1, 2, 3, 4, 1, 2, 5, 1, 2, 3, 4, 5 (4 frames)

Ref	F1	F2	F3	F4	Fault
1	1	-	-	-	F
2	1	2	-	-	F
3	1	2	3	-	F
4	1	2	3	4	F
1	1	2	3	4	
2	1	2	3	4	

Optimal (OPT/MIN) Algorithm

The theoretically optimal algorithm

- Replace the page that **will not be used for the longest time** in the future
- Guarantees the **lowest page fault rate** among all algorithms
- **Belady's Anomaly does not occur**
- Problem: **impossible to implement in practice** (cannot know future references)
- Used as a performance comparison baseline (upper bound) for other algorithms

Optimal Algorithm Operation Example

3 frames, reference string: 7 0 1 2 0 3 0 4 2 3 0 3 2 1 2 0 1 7 0 1

Ref	F1	F2	F3	Fault	Replacement reason
7	7	-	-	F	
0	7	0	-	F	
1	7	0	1	F	
2	**2**	0	1	F	7 used at #18 (farthest)
0	2	0	1		
3	2	0	**3**	F	1 used at #14 (farthest)

Optimal Algorithm Operation Example (cont.)

Ref	F1	F2	F3	Fault
3	2	0	3	
2	2	0	3	
1	2	**1**	3	F
2	2	1	3	
0	2	1	3	
1	2	1	3	
7	**7**	1	3	

LRU (Least Recently Used)

Replace the page that has not been used for the longest time

- Approximation of Optimal: judge based on **the past instead of the future**
- Key assumption: a page not used recently will likely not be used in the future
- **Belady's Anomaly does not occur** (Stack algorithm)
- Equivalent to applying OPT in reverse time direction

LRU Operation Example

3 frames, reference string: 7 0 1 2 0 3 0 4 2 3 0 3 2 1 2 0 1 7 0 1

Ref	F1	F2	F3	Fault	Replacement reason
7	7	-	-	F	
0	7	0	-	F	
1	7	0	1	F	
2	**2**	0	1	F	7 was used least recently
0	2	0	1		
3	2	0	**3**	F	1 was used least recently

LRU Operation Example (cont.)

Ref	F1	F2	F3	Fault
0	**0**	3	2	F
3	0	3	2	
2	0	3	2	
1	**1**	3	2	F
2	1	3	2	
0	1	**0**	2	F
1	1	0	2	

Algorithm Comparison Summary

Same reference string, 3 frames

Algorithm	Page Faults	Characteristics
FIFO	**15**	Simple but has Belady's Anomaly
LRU	**12**	Good performance, high implementation cost
OPT	**9**	Optimal but impossible to implement

LRU Implementation Methods

Hardware support is essential

- **Method 1: Counter (time recording)**
  - Record last access time (time-of-use) in each page table entry
  - Add a logical clock to CPU, increment on every memory access
  - Select the page with the smallest time value for replacement
  - Disadvantage: requires searching the entire table, memory write on every access
- **Method 2: Stack (doubly linked list)**
  - Move the page to the top of the stack on access
  - The bottom of the stack is the LRU page (tail pointer)
  - Determine the victim in O(1) without searching
  - Disadvantage: up to 6 pointer changes on every access

Stack Algorithms and Belady's Anomaly

Why LRU and OPT do not suffer from Belady's Anomaly

- **Stack algorithm**: the set of pages in n frames is always a **subset** of the set of pages in n+1 frames
- LRU: n frames = the n most recently referenced pages
  - With n+1 frames, these n pages are still included
- Adding frames -> page faults never increase
- FIFO is not a stack algorithm -> Belady's Anomaly is possible

Part 3

LRU Approximation Algorithms

Need for LRU-Approximation

Most hardware does not support true LRU

- True LRU: requires counter or stack update on every memory access
  - Implementing via software interrupt would be 10x slower or more
- What most hardware provides: **reference bit**
  - Hardware automatically sets bit = 1 when a page is referenced
  - Can tell which pages were used, but **not the order**
- Use the reference bit to **approximate** LRU

Additional-Reference-Bits Algorithm

Recording the history of reference bits

- Maintain an 8-bit byte (shift register) for each page
- At each periodic timer interrupt (e.g., 100ms):
  1. Shift-in the current reference bit as the **most significant bit** of the byte
  2. Shift remaining bits to the right
  3. Clear the reference bit

```
Example: 8-bit history
11000100 → Used 2 times in recent 3 intervals, then 1 time after
01110111 → Used 1 time then 3 consecutive + 3 consecutive uses
→ 11000100 was used more recently (unsigned comparison)
```

- The page with the smallest byte value is closest to LRU
- Pages with equal values are selected by FIFO

Second-Chance (Clock) Algorithm

Combining FIFO + reference bit

- Select the page replacement candidate in basic FIFO order
- Check the **reference bit** before replacement
  - bit = 0 -> **replace** the page
  - bit = 1 -> change bit to 0 and **grant a second chance**
    - Reset arrival time to current time
    - Move to the next FIFO page
- Implemented as a **circular queue** -> **Clock Algorithm**
- Pointer circulates like a clock hand searching for a victim
- Worst case: if all bits = 1, traverse one full cycle resetting all to 0 then replace (= FIFO)
- Frequently used pages always have reference bit = 1 -> never replaced

Clock Algorithm Operation Visualization

Victim search process in a circular queue

```
[Initial state]                [After search]
   v (pointer)                    v (pointer, new victim)
+------+                     +------+
| P1: 1| → clear to bit=0   | P1: 0|
| P2: 1| → clear to bit=0   | P2: 0|
| P3: 0| → victim!          | NEW  | ← new page inserted
| P4: 1|                     | P4: 1|
| P5: 0|                     | P5: 0|
+------+                     +------+

P1, P2 survive by receiving a second chance
P3 is replaced because reference bit = 0
```

Enhanced Second-Chance Algorithm

Combining Reference bit + Modify bit

- Classify into **4 classes** using (reference bit, modify bit) pairs

Class	(R, M)	Meaning	Replacement priority
1	(0, 0)	Not recently used, not modified	**Highest priority**
2	(0, 1)	Not recently used, modified	2nd priority (write back needed)
3	(1, 0)	Recently used, not modified	3rd priority
4	(1, 1)	Recently used, modified	**Lowest priority**

- Replace pages belonging to the lowest class
- Replacing a modified page requires additional I/O, hence lower priority

Counting-Based Algorithms

Algorithms based on reference counts

- **LFU (Least Frequently Used)**
  - Replace the page with the **fewest** references
  - Problem: pages heavily used early but unused later remain
  - Solution: periodically right-shift the count (exponential decay averaging)
- **MFU (Most Frequently Used)**
  - Replace the page with the **most** references
  - Logic: pages with small counts just arrived and haven't been used yet
- Neither generally approximates OPT well
- High implementation cost, rarely used in practice

Page Buffering Algorithms

Techniques to complement page replacement

- **Maintain a free frame pool**
  - Allocate immediately from free frames before selecting a victim on page fault
  - Process can restart quickly
  - Victim page is written back later
- **Modified page list**
  - Collect dirty pages and write back in batch (batch I/O)
  - Write in advance when the paging device is idle
- **Retain previous page contents in free frames**
  - Keep page information in the free frame pool
  - If needed again, recover immediately without I/O
  - Used in UNIX-based systems together with second-chance

Part 4

Frame Allocation / Thrashing

Frame Allocation Overview

The problem of allocating frames to processes

- How to distribute total available frames among multiple processes?
- **Minimum allocation**: must guarantee a minimum number of frames
  - Determined by the maximum number of pages referenced by a single instruction
  - More frames needed if indirect addressing is used
  - Example: 1 level of indirect addressing -> minimum 3 frames
- Minimum frame count: determined by **computer architecture**
- Maximum frame count: determined by **available physical memory**

Equal vs Proportional Allocation

Frame distribution strategies

- **Equal Allocation**
  - Allocate the same number of frames to all processes
  - m frames, n processes -> m/n each
  - Example: 93 frames, 5 processes -> 18 frames each (3 spare)
- **Proportional Allocation**
  - Allocate frames proportionally to process size
  - a_i = (s_i / S) x m (s_i = process size, S = sum, m = available frames)
  - Example: 62 frames, Process A (10KB), B (127KB)
    - A: 10/137 x 62 = approximately 4 frames
    - B: 127/137 x 62 = approximately 57 frames
  - **Priority-based allocation**: can also be based on priority instead of size

Global vs Local Replacement

Scope of frames eligible for replacement

- **Global Replacement**
  - Select victim from **all** frames
  - Can take frames from other processes
  - Better **throughput** -> generally used
  - Disadvantage: a process cannot control its own page fault rate
- **Local Replacement**
  - Select victim only from frames **allocated to itself**
  - Does not affect other processes
  - Page fault rate is predictable
  - Disadvantage: may not utilize available memory efficiently

Reclaiming Pages

Practical implementation strategy for global replacement

- Page reclaiming starts when the free-frame list drops below a **minimum threshold**
- **Reaper** routine reclaims pages from all processes
  - Uses LRU approximation algorithm
  - Stops when free memory reaches the **maximum threshold**

```
free memory
    |
    |    +- maximum threshold ————————
    |    |  d: reaper stops
    |    |
    |    |  b: reaper stops
    |    |
    |    +- minimum threshold ————————
    |       a: reaper starts   c: reaper starts
    +————————————————————————————————————— time
```

- Linux: if free memory is critically low, **OOM Killer** runs
  - Terminates the process with the highest OOM score

NUMA and Frame Allocation

Considerations in Non-Uniform Memory Access environments

- NUMA system: each CPU has local memory
- Accessing memory closer to the same CPU is faster
- On page fault, allocate a frame **close to the CPU running the process**
- Linux: maintains a separate free-frame list for each NUMA node
- Scheduler attempts to schedule processes on the CPU where they previously ran
  - Improved cache hits + reduced memory access time

Thrashing

A phenomenon where most time is spent handling page faults

- Occurs when a process **does not have enough frames**
- Definition: spending more time on page fault handling than on execution
- Vicious cycle process
  1. Insufficient frames for the process -> frequent page faults
  2. I/O surges due to page fault handling
  3. CPU utilization drops (processes waiting for I/O)
  4. OS **loads more processes** to increase CPU utilization
  5. Each process has even fewer frames
  6. Page faults increase further -> **repeats (vicious cycle)**

Consequences of Thrashing

Relationship between CPU utilization and Degree of Multiprogramming

- Increasing multiprogramming degree -> CPU utilization **initially increases**
- Beyond a certain point -> CPU utilization **drops sharply**
- This turning point is where **thrashing** begins

```
CPU utilization
    ^
    |       ^
    |      /  \
    |     /    \ ← thrashing begins
    |    /      \
    |   /        \
    |  /          \
    | /            \
    +──────────────────────→ degree of multiprogramming
```

- Solution: use local replacement or priority replacement
  - Prevent one process's thrashing from affecting other processes

Locality Model

Theoretical basis for preventing thrashing

- A process **moves from locality to locality** during execution
- **Locality**: a set of pages used intensively over a period of time
- A new locality is defined on function calls
  - Function instructions, local variables, some global variables
  - Leaves that locality when the function returns
- Localities can **overlap** with each other
- If enough frames are allocated to accommodate the current locality:
  - Faults occur only until all pages of the locality are loaded
  - No faults until the locality changes
- If the locality cannot be accommodated -> **thrashing**

Working-Set Model

A practical model for preventing thrashing

- **Working-Set Window (Delta)**
  - The set of pages used in the most recent Delta memory references
  - If Delta is too small, it cannot contain the locality
  - If Delta is too large, multiple localities overlap
  - If Delta = infinity, all pages ever used by the process
- Example: Delta = 10

```
...2 6 1 5 7 7 7 7 5 1 | 6 2 3 4 1 2 3 4 4 4 | 3 4 3 4...
                    t1                     t2
WS(t1) = {1, 2, 5, 6, 7}    WS(t2) = {3, 4}
```

Working-Set Size and Thrashing Prevention

Resource management using WSS

- **WSS_i** = Working-Set Size of process i (number of pages)
- **D** = sum of WSS of all processes (total frame demand)
- D > m (available frames) -> **thrashing occurs!**
- Working-Set Strategy
  - OS monitors the working set of each process
  - Allocate WSS_i frames to each process
  - If D <= m: spare frames can be used to load new processes
  - If D > m: **suspend** one process to free up frames
  - Result: thrashing prevention + CPU utilization optimization

Working-Set Implementation

Using timer interrupts and reference bits

- Accurate Working-Set tracking is expensive
- Approximate implementation: **fixed-interval timer interrupt + reference bit**

```
Delta = 10,000 references
Timer interrupt: every 5,000 references

- On timer interrupt:
  1. Copy current reference bit value to in-memory bit
  2. Clear the reference bit

- On page fault:
  - Check current reference bit + 2 in-memory bits
  - If any is 1 → used within the last 10,000~15,000 references
  - If all are 0 → not included in the working set

- Increasing the number of history bits improves accuracy, but increases interrupt cost
```

Page-Fault Frequency (PFF)

Direct thrashing control

- **Directly monitor** the page fault rate
- Set upper bound and lower bound

Situation	Action
PFF > upper bound	**Allocate more frames** to the process
PFF < lower bound	**Reclaim frames** from the process
PFF > upper bound + no available frames	**Suspend** the process

- Advantages over Working-Set
  - Simpler to implement
  - **Direct** page fault rate control

PFF Graph

Relationship between number of frames and page fault rate

```
page fault rate
    ^
    |\
    | \
    |  \  ← increase frames (PFF > upper bound)
    |   \
    |    -- upper bound ———————————————
    |      \
    |       \
    |        -- lower bound ————————————
    |          \ ← decrease frames (PFF < lower bound)
    |           \__
    +————————————————————————————————————→ number of frames
```

- Key: manage to operate stably between upper/lower bounds

Part 5

Memory Compression / Kernel Memory

Memory Compression

An alternative to swap out

- Traditional method: swap out modified pages to **disk**
- Memory Compression: instead of swapping out, **compress** multiple pages and keep them in memory
  - Store compressed pages in a single frame
  - Improved performance by reducing disk I/O
  - Especially useful on mobile devices (no swap space)
- Operating systems that use it: macOS, iOS, Android, Windows 10 and later
- Apple WKdm, Microsoft Xpress: compress pages to 30~50% of original size
- When a compressed page is referenced again -> page fault -> decompress
- Compression is faster than paging to SSD

Memory Compression Operation

Free-frame list and compressed frame list

```
[Before compression]
free-frame list:  7 → 2 → 9 → 21 → 27 → 16
modified list:    15 → 3 → 35 → 26

[After compression]
- Take frame 7 from the free-frame list
- Compress contents of frames 15, 3, 35 and store in frame 7
- Frame 7 moves to the compressed frame list
- Frames 15, 3, 35 are returned to the free-frame list

free-frame list:  2 → 9 → 21 → 27 → 16 → 15 → 3 → 35
compressed list:  7 (contains contents of 15+3+35)
modified list:    26
```

Kernel Memory Allocation

Kernel memory is managed separately from user memory

- Reasons for managing kernel memory separately

  1. The kernel requires allocation of various sizes of data structures
     - Sizes smaller than a page are frequent -> need to minimize fragmentation
  2. Some hardware requires **contiguous physical memory** (DMA)
     - Page-unit allocation does not guarantee contiguity

- Two strategies

  - **Buddy System**
  - **Slab Allocation**

Buddy System

Memory allocation by splitting into powers of 2

- Allocate from a contiguous physical page region
- Round up the request size to the **next power of 2**
  - Example: 21KB request -> 32KB allocation
- Splitting process (256KB segment, 21KB request)

```
256KB
+-- 128KB (AL)          ← not used
+-- 128KB (AR)
    +-- 64KB (BL)
    |   +-- 32KB (CL)  ← allocated for 21KB request!
    |   +-- 32KB (CR)  ← buddy
    +-- 64KB (BR)       ← not used
```

- **Coalescing**: merge with adjacent buddy on deallocation
  - CL freed -> CL + CR = 64KB -> BL + BR = 128KB -> 256KB restored
- Advantage: coalescing is fast and simple
- Disadvantage: **internal fragmentation** (33KB request -> 64KB allocation, approximately 50% waste)

Slab Allocation

Per-kernel-object cache management

- **Slab**: one or more contiguous physical pages
- **Cache**: composed of one or more slabs, dedicated to a specific kernel object type
- Each cache stores **instances of a specific data structure**
- Examples: process descriptor cache, file object cache, semaphore cache
- How it works
  - Kernel object creation request -> allocate a **free object** from the corresponding cache
  - Object deallocation -> return to cache's free list (reuse)
- Slab states: **full** / **partial** / **empty**
  - Try allocating from partial slabs first
  - If none, from empty slabs; if none, allocate a new slab

Slab Allocation Advantages

- **Minimized fragmentation**
  - Allocation exactly matches object size (no internal fragmentation)
  - Each cache is dedicated to specific objects, so management is seamless
- **Fast allocation/deallocation**
  - Reuses pre-initialized objects
  - No initialization cost unlike malloc/free
- Slab-related allocators in the Linux kernel
  - **SLAB**: original Linux implementation
  - **SLOB**: for memory-constrained embedded systems (simple list of blocks)
  - **SLUB**: default allocator since Linux 2.6.24
    - Less metadata overhead than SLAB
    - Better performance in multi-CPU environments

Part 6

Other Considerations

Prepaging

Strategy to reduce initial page faults

- Problem with pure demand paging: many page faults at startup
- **Prepaging**: preload pages that are expected to be needed
  - Remember the working set when a process is suspended
  - Preload the entire working set on resume
- Effectiveness analysis: prepage s pages, alpha fraction is actually used
  - s x alpha = saved page faults
  - s x (1 - alpha) = unnecessarily loaded pages
  - Beneficial if alpha is close to 1, detrimental if close to 0
- Linux **readahead()**: preload file contents into memory

Program Structure and Locality

How programmers can reduce page faults

- 128 x 128 array, page size = 128 words (one row = one page)

```
// Bad example: column-major access → different page on every access
int data[128][128];
for (int j = 0; j < 128; j++)
    for (int i = 0; i < 128; i++)
        data[i][j] = 0;  // 128 x 128 = 16,384 page faults!
```

```
// Good example: row-major access → sequential access one page at a time
int data[128][128];
for (int i = 0; i < 128; i++)
    for (int j = 0; j < 128; j++)
        data[i][j] = 0;  // 128 page faults only!
```

- **Stack**: good locality (always accesses top)
- **Hash table**: poor locality (references are distributed)

I/O Interlock and Page Locking

Protecting pages during I/O operations

- Problem scenario: Process A is using an I/O buffer page
  - The page is replaced via global replacement
  - When I/O completes, data is written to another process's page!
- Solution 1: Indirect I/O through system memory
  - Always copy to kernel memory first -> then move to user memory
  - Can have overhead
- Solution 2: **Lock bit (Pin)**
  - Set lock bit on frame -> excluded from replacement candidates
  - Unlock after I/O completion
  - OS kernel is also locked to prevent page out

Lab

Page Replacement Algorithm Simulator

Lab – Page Replacement Algorithm Simulator

Comparing FIFO, LRU, and Optimal

Objectives

- Simulate three algorithms given a reference string and number of frames as input
- Compare page fault counts for each algorithm
- Output the page fault process step by step

Implementation Requirements

- Input: reference string (integer array), number of frames
- Output frame state for each reference for each algorithm
- Mark when a page fault occurs
- Final output: comparison table of total page faults per algorithm

Input Example

```
Reference String: 7 0 1 2 0 3 0 4 2 3 0 3 2 1 2 0 1 7 0 1
```

Lab – Core Implementation Logic

```python
def fifo(ref_string, num_frames):
    frames = []
    faults = 0
    for page in ref_string:
        if page not in frames:
            faults += 1
            if len(frames) < num_frames:
                frames.append(page)
            else:
                frames.pop(0)       # Remove the oldest page
                frames.append(page)
        print(f"Ref: {page}  Frames: {frames}  {'* Fault' if page not in frames else ''}")
    return faults
```

Lab – Output Example

Step-by-step frame state and page fault comparison

```
═══ FIFO Algorithm ═══
Ref: 7  Frames: [7, -, -]  * Page Fault
Ref: 0  Frames: [7, 0, -]  * Page Fault
Ref: 1  Frames: [7, 0, 1]  * Page Fault
Ref: 2  Frames: [2, 0, 1]  * Page Fault
Ref: 0  Frames: [2, 0, 1]
Ref: 3  Frames: [2, 3, 1]  * Page Fault
 ...
Total Page Faults: 15

═══ Comparison ═══
| Algorithm | Page Faults |
├──────────┼────────────┤
| FIFO      | 15          |
| LRU       | 12          |
| Optimal   | 9           |
```

Summary

Key takeaways for this week

- **Virtual Memory**: provides address space larger than physical memory, separates logical/physical
- **Demand Paging**: loads pages only when needed, handles page faults
- **Copy-on-Write**: copies pages only on write during fork()
- **Page Replacement**: FIFO (Belady's Anomaly), Optimal, LRU, Second-Chance
- **Frame Allocation**: Equal, Proportional, Global vs Local
- **Thrashing**: insufficient frames -> page fault explosion -> CPU utilization plummets
- **Working-Set Model, PFF** to prevent thrashing
- **Memory Compression**: compress instead of swap out
- **Kernel Memory**: Buddy System (coalescing), Slab Allocation (object cache)

Next Week Preview

Next Week Preview: Mass-Storage Structure (Ch 11)

- Disk Scheduling Algorithms (FCFS, SCAN, C-SCAN, LOOK)
- RAID (Redundant Arrays of Inexpensive Disks)
- Storage Device Management